Rapport individuel – Louay MASBAH

Niveau organisation

Dans la dernière itération, j'ai essayé d'ajouter du multithreading pour régler quelques problèmes dans le jeu, mais cela a compliqué beaucoup le travail. On a donc choisi d'annuler le multithreading.

Nous avons réparti les tâches : Mohammed Anis et moi — lui s'occupait de compléter ce qui manquait dans l'éditeur de niveaux (placement des obstacles et enregistrement de niveau), et moi je m'occupais d'offrir à l'utilisateur la possibilité de choisir un niveau à jouer. J'ai également réglé le problème du bouton « Tester le niveau » qui ne faisait rien. Il restait quelques bugs que j'ai demandé à Chada de régler, et elle l'a fait.

J'ai aussi débogué quelques petits problèmes : ajout d'un constructeur dans Joueur, mise du budget initial à 1000 si non spécifié, et d'autres petites modifications non détaillées dans ce rapport.

Sélection de map et lancement du jeu

Implémentation d'un écran de sélection de map (SelectionMap.java) intercalé entre le menu principal et le jeu. L'écran scanne automatiquement le répertoire src/core/maps/ et liste tous les fichiers .txt disponibles. L'utilisateur peut soit choisir une map manuellement, soit en lancer une aléatoirement via un bouton dédié.

- Scan dynamique du répertoire de maps au lancement.
- Bouton « Map aléatoire » pour une sélection au hasard.
- Style visuel cohérent avec le reste de l'interface (fond animé, curseur personnalisé, boutons arrondis).

Refonte de Game.java

Ajout de deux nouveaux constructeurs à Game.java afin de supporter les nouveaux flux de jeu sans casser la compatibilité existante.

- Game(Joueur, Joueur, String cheminMap) : charge n'importe quel fichier .txt de map.
- Game(Joueur, Joueur, Map) : accepte une map déjà construite en mémoire, utilisé par l'éditeur de niveaux pour le test.

Éditeur de niveaux – test et sauvegarde

Participation à l'amélioration de l'éditeur de niveaux, en collaboration avec le code existant basé sur le pattern MVC (ModeEditeur / ControleurEditeur / AfficherGrille).

Test du niveau depuis l'éditeur

Branchement du bouton « Tester le niveau » : la map courante (obstacles, position de départ, cible) est construite en mémoire et lancée dans FenetreJeu. À la fermeture du test, l'éditeur reprend la main grâce à un callback Runnable passé à FenetreJeu.

Après cela, on pouvait remarquer que le jeu se lançait bien, mais sans les obstacles placés dans l'éditeur, et on ne pouvait pas retourner à l'éditeur après — seulement au MenuPrincipal. Ces problèmes ont été réglés par Chada.

Placement multi-cases des murs

Correction du placement des obstacles de type Mur. Auparavant limité à une seule case, le mur peut désormais être étalé sur plusieurs cases par un glisser-déposer. Le contrôleur capture le point de début (mousePressed) et le point de fin (mouseReleased), tous deux snappés à la grille. La taille minimale d'une case est garantie en cas de simple clic.

Correction de la sauvegarde des murs

Correction d'un bug critique : les obstacles Mur étaient sauvegardés avec seulement 2 coordonnées (x1, y1) au lieu des 4 requises par ChargeurMap (x1, y1, x2, y2), rendant les maps illisibles au chargement. Correction réalisée à trois niveaux : stockage du point de fin dans AfficherGrille (finsMurs), exposition de ce point via ObstacleMur.setPointFinal(), et écriture correcte des 4 coordonnées dans sauvegarderMapDansFichier(). La méthode testerNiveau() a également été corrigée pour utiliser les vraies dimensions du mur au lieu d'une valeur fixe arbitraire.

Modification de FenetreJeu

Ajout d'un constructeur avec callback Runnable permettant de spécifier une action au clic sur « Retour ». Lorsque la fenêtre de jeu est ouverte depuis l'éditeur, ce callback réaffiche l'éditeur au lieu d'ouvrir le menu principal. Un bandeau « MODE TEST » est affiché et le bouton retour est renommé « Retour Éditeur » pour indiquer clairement le contexte à l'utilisateur.

Utilisation de l'IA

Comme dans la dernière itération, j'ai utilisé Claude. Comme Claude n'avait pas tout le contexte, j'avais besoin de lui expliquer ce qu'il fallait écrire pour économiser le temps de rédaction du code. Son code causait toujours des erreurs et j'ai fait le débogage moi-même pour préserver les tokens. On peut donc dire que le cœur de mon code a été écrit par l'IA : je l'ai lu afin de le comprendre, de vérifier qu'il réalisait bien la tâche souhaitée, et je l'ai intégré moi-même dans le code général du projet en modifiant plusieurs lignes.

J'ai aussi utilisé l'IA pour rendre ce rapport plus compréhensible et grammaticalement correct.